

MailExchangeService

Abstract base class — shared outbound-mail contract

EWS (Exchange Web Services)



Microsoft Graph API

notification-service · com.dbs.edoc.notification.services.notification.ews · Spring Boot / Java 17

Prepared ahead of the Exchange Online EWS retirement (EWS blocking begins 1 Oct 2026, full shutdown 1 Apr 2027)

1. Role of this Class

MailExchangeService is the shared abstract parent for **CloudMailExchangeService** (Exchange Online / Office 365 mailboxes) and **InternalMailExchangeService** (on-premises Exchange mailboxes). It owns the one piece of logic both subclasses need: turning a `Mail` domain object into an outbound message and filing a copy in the sender's **Sent Items** folder. Each subclass only supplies *how* to authenticate and *which client object* talks to the mail server — that is exactly the part this migration changes.

Why this class has to change

Exchange Web Services (EWS) for Exchange Online will be blocked for tenants starting **1 October 2026** and permanently shut down by **1 April 2027** (Microsoft's official EWS retirement announcement). The entire `microsoft.exchange.webservices.data.*` object model used here — `EmailMessage`, `WellKnownFolderName`, `ExtendedPropertyDefinition`, `MessageBody` — is EWS-only and has no future on Exchange Online. Microsoft's recommended, near-feature-complete replacement is the **Microsoft Graph API** (`/users/{id}/sendMail`), accessed here through the official **microsoft-graph** Java SDK.

2. Before Migration — EWS-based Implementation

MailExchangeService.java – CURRENT (EWS, deprecated path)

```
package com.dbs.edoc.notification.services.notification.ews;

import com.dbs.edoc.crypto.Crypter;
import com.dbs.edoc.crypto.CryptoUtilException;
import com.dbs.edoc.crypto.impl.DefaultCrypter;
import com.dbs.edoc.notification.error.ExchangeMessageException;
import com.dbs.edoc.notification.services.notification.Mail;
import microsoft.exchange.webservices.data.core.ExchangeService;
import microsoft.exchange.webservices.data.core.enumeration.property.MapiPropertyType;
import microsoft.exchange.webservices.data.core.enumeration.property.WellKnownFolderName;
import microsoft.exchange.webservices.data.core.service.item.EmailMessage;
import microsoft.exchange.webservices.data.property.complex.EmailAddress;
import microsoft.exchange.webservices.data.property.complex.MessageBody;
import microsoft.exchange.webservices.data.property.definition.ExtendedPropertyDefinition;
import org.apache.commons.lang3.StringUtils;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import java.util.List;

public abstract class MailExchangeService {

    private static final Logger LOGGER = LoggerFactory.getLogger(MailExchangeService.class);
    private final Crypter crypter = DefaultCrypter.getAesInstance();

    abstract void initializeExchangeService(String username, String password, String domain) throws ExchangeMessageException;

    abstract ExchangeService getExchangeService();

    public void saveMailToOutbox(String senderEmail, List<String> emailPwdPair, Mail mail, byte[] attachment, String fileName) throws ExchangeMessageException, CryptoUtilException {
        if (emailPwdPair != null) {
            LOGGER.info("Domain and Password Pair found for [{}]", senderEmail);
            String safePwdKey = getPwdForEmail(emailPwdPair.get(0).trim());

            try {
                initializeExchangeService(senderEmail.trim(), safePwdKey, emailPwdPair.get(1).trim());
            } catch (Exception em) {
                LOGGER.error("Exception while initializing exchange service for [{}]", senderEmail);
                throw new ExchangeMessageException("Exception while initializing exchange service", em);
            }

            try {
                EmailMessage exchangeMessage = new EmailMessage(getExchangeService());
                for (String toAddress : mail.getTo()) {
                    exchangeMessage.getToRecipients().add(toAddress);
                }

                for (String ccAddress : mail.getCc()) {
                    exchangeMessage.getCcRecipients().add(ccAddress);
                }

                for (String bccAddress : mail.getBcc()) {
                    exchangeMessage.getBccRecipients().add(bccAddress);
                }

                if (attachment != null && !StringUtils.isBlank(fileName)) {
                    exchangeMessage.getAttachments().addFileAttachment(fileName, attachment);
                }

                ExtendedPropertyDefinition messageReadFlag = new ExtendedPropertyDefinition(3591, MapiPropertyType.Integer);
                exchangeMessage.setExtendedProperty(messageReadFlag, 1);
                exchangeMessage.setSubject(mail.getSubject());
                exchangeMessage.setBody(MessageBody.getMessageBodyFromText(mail.getBody()));
                exchangeMessage.setFrom(new EmailAddress(senderEmail));
                LOGGER.info("Attempting to save Email [{}] to the email box of [{}]", mail.getSubject(), senderEmail);
                exchangeMessage.save(WellKnownFolderName.SentItems);
                LOGGER.info("Mail [{}] Saved in the Sent items Folder of [{}]", mail.getSubject(), senderEmail);
            } catch (Exception e) {
                LOGGER.info("Error while saving to Sent items for [{}]", mail.getSubject());
                LOGGER.error(" Details of failure : ", e);
                throw new ExchangeMessageException("Error occurred while saving the message to Sent Items of [" + mail.getSubject() + "]", e);
            }
        } else {
            LOGGER.warn("No Credentials information found for [{}]. Can not save the email in to Sent Items", senderEmail);
        }
    }

    private String getPwdForEmail(String password) throws CryptoUtilException {
        return crypter.decrypt(password);
    }
}
```

3. After Migration — Microsoft Graph Implementation

MailExchangeService.java – MIGRATED (Microsoft Graph)

```
package com.dbs.edoc.notification.services.notification.ews;

import com.dbs.edoc.crypto.Crypter;
import com.dbs.edoc.crypto.CryptoUtilException;
import com.dbs.edoc.crypto.impl.DefaultCrypter;
import com.dbs.edoc.notification.error.ExchangeMessageException;
import com.dbs.edoc.notification.services.notification.Mail;
import com.microsoft.graph.models.Attachment;
import com.microsoft.graph.models.BodyType;
import com.microsoft.graph.models.EmailAddress;
import com.microsoft.graph.models.FileAttachment;
import com.microsoft.graph.models.ItemBody;
import com.microsoft.graph.models.Message;
import com.microsoft.graph.models.Recipient;
import com.microsoft.graph.serviceclient.GraphServiceClient;
import com.microsoft.graph.users.item.sendmail.SendMailPostRequestBody;
import org.apache.commons.lang3.StringUtils;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import java.util.ArrayList;
import java.util.List;

public abstract class MailExchangeService {

    private static final Logger LOGGER = LoggerFactory.getLogger(MailExchangeService.class);
    private final Crypter crypter = DefaultCrypter.getAesInstance();

    // Signature is unchanged on purpose -- CloudMailExchangeService and
    // InternalMailExchangeService only need to change what happens *inside*
    // this method, callers of saveMailToOutbox(...) do not change at all.
    abstract void initializeExchangeService(String username, String password, String domain) throws ExchangeMessageException;

    // Replaces "abstract ExchangeService getExchangeService()".
    // Each subclass now hands back a configured GraphServiceClient instead
    // of an EWS ExchangeService.
    abstract GraphServiceClient getGraphServiceClient();

    public void saveMailToOutbox(String senderEmail, List<String> emailPwdPair, Mail mail, byte[] attachment, String fileName) throws
ExchangeMessageException, CryptoUtilException {
        if (emailPwdPair != null) {
            LOGGER.info("Domain and Password Pair found for [{}]", senderEmail);
            String safePwdKey = getPwdForEmail(emailPwdPair.get(0).trim());

            try {
                initializeExchangeService(senderEmail.trim(), safePwdKey, emailPwdPair.get(1).trim());
            } catch (Exception em) {
                LOGGER.error("Exception while initializing exchange service for [{}]", senderEmail);
                throw new ExchangeMessageException("Exception while initializing exchange service", em);
            }

            try {
                Message message = new Message();
                message.setSubject(mail.getSubject());

                ItemBody body = new ItemBody();
                body.setContentType(BodyType.Text);
                body.setContent(mail.getBody());
                message.setBody(body);

                message.setToRecipients(toRecipients(mail.getTo()));
                message.setCcRecipients(toRecipients(mail.getCc()));
                message.setBccRecipients(toRecipients(mail.getBcc()));

                if (attachment != null && !StringUtils.isBlank(fileName)) {
                    FileAttachment fileAttachment = new FileAttachment();
                    fileAttachment.setOdataType("#microsoft.graph.fileAttachment");
                    fileAttachment.setName(fileName);
                    fileAttachment.setContentBytes(attachment);
                    List<Attachment> attachments = new ArrayList<>();
                    attachments.add(fileAttachment);
                    message.setAttachments(attachments);
                }

                LOGGER.info("Attempting to send Email [{}] from the mailbox of [{}]", mail.getSubject(), senderEmail);

                // Graph's sendMail action both sends the message AND (when
                // saveToSentItems = true) writes it to the Sent Items folder
                // in one call -- the separate ".save(SentItems)" step from
                // EWS is no longer necessary.
                SendMailPostRequestBody sendMailBody = new SendMailPostRequestBody();
                sendMailBody.setMessage(message);
                sendMailBody.setSaveToSentItems(true);

                getGraphServiceClient()
                    .users()
                    .byUserId(senderEmail)
                    .sendMail()
```

```

        .post(sendMailBody);

        LOGGER.info("Mail [{}] Sent and Saved in the Sent Items Folder of [{}]", mail.getSubject(), senderEmail);
    } catch (Exception e) {
        LOGGER.info("Error while saving to Sent items for [{}]", mail.getSubject());
        LOGGER.error(" Details of failure : ", e);
        throw new ExchangeMessageException("Error occurred while saving the message to Sent Items of [" + mail.getSubject() + "] ",
e);
    }
} else {
    LOGGER.warn("No Credentials information found for [{}]. Can not save the email in to Sent Items", senderEmail);
}
}

private List<Recipient> toRecipients(List<String> addresses) {
    List<Recipient> recipients = new ArrayList<>();
    if (addresses != null) {
        for (String address : addresses) {
            Recipient recipient = new Recipient();
            EmailAddress emailAddress = new EmailAddress();
            emailAddress.setAddress(address);
            recipient.setEmailAddress(emailAddress);
            recipients.add(recipient);
        }
    }
    return recipients;
}

private String getPwdForEmail(String password) throws CryptoUtilException {
    return crypter.decrypt(password);
}
}

```

4. What Actually Changed, and Why

Concern	Before (EWS)	After (Microsoft Graph)
Client object subclasses must provide	<code>abstract ExchangeService getExchangeService()</code>	<code>abstract GraphServiceClient getGraphServiceClient()</code>
Building the message	<code>new EmailMessage(exchangeService)</code> , then mutate recipients/body on the object itself	Plain Graph Message POJO built independently of the client, then handed to the send call
Recipients	<code>exchangeMessage.getToRecipients().add(String)</code> — EWS accepts raw address strings	Graph needs a Recipient wrapping an EmailAddress object per address — small helper <code>toRecipients()</code> added
Attachments	<code>exchangeMessage.getAttachments().addFileAttachment(name, bytes)</code>	FileAttachment object with <code>@odata.type</code> set, added to a <code>List<Attachment></code>
Sending + filing in Sent Items	Two implicit steps: EWS builds a <i>draft</i> and <code>.save(WellKnownFolderName.SentItems)</code> only saves it — the app never actually calls "send" separately in this code path	One call: <code>users().byUserId(senderEmail).sendMail().post(...)</code> with <code>saveToSentItems(true)</code> — Graph sends and files the copy for you
"Read" flag / MAPI property	<code>ExtendedPropertyDefinition(3591, MapiPropertyType.Integer)</code> — raw MAPI property manipulation	Not needed: Graph's <code>sendMail</code> action already marks the sent copy as read the same way Outlook does; if a future need for custom MAPI-style properties arises, Graph exposes <code>singleValueExtendedProperties</code> using the same property IDs
Password decryption / crypto	<code>DefaultCrypter.getAesInstance().decrypt(...)</code>	Unchanged — kept as-is (see note below on why this hook is now optional per-subclass)
Public method signature	<code>saveMailToOutbox(senderEmail, emailPwdPair, mail, attachment, fileName)</code>	Identical — zero changes required in any caller/controller class

Fresher-friendly note

Notice the public method name `saveMailToOutbox` and its signature never change. That is a deliberate design goal of this migration: every controller, scheduler, or service that already calls `mailExchangeService.saveMailToOutbox(...)` keeps working untouched. All the EWS-specific plumbing is private to this class and its two subclasses.

5. Maven Dependency Changes (pom.xml)

Remove (EWS stack)	Add (Microsoft Graph stack)
<pre><dependency> <groupId>com.microsoft.ews-java-api</groupId> <artifactId>ews-java-api</artifactId> </dependency> <dependency> <groupId>com.microsoft.aad</groupId> <artifactId>adal4j</artifactId> </dependency></pre>	<pre><dependency> <groupId>com.microsoft.graph</groupId> <artifactId>microsoft-graph</artifactId> <version>6.20.0</version> </dependency> <dependency> <groupId>com.azure</groupId> <artifactId>azure-identity</artifactId> <version>1.13.3</version> </dependency></pre>

Version check

Confirm the latest published microsoft-graph and azure-identity versions on Maven Central before locking these in a real pom.xml — both SDKs release frequently.